Study Material 2

Read Chapters 3 of Database System Concepts and answer the following questions.

1. Fill in the blanks.

(a) The (SQL DDL) provides commands for defining relation schemas, deleting relations, and modifying relation schemas.

(b) The (SQL DML) provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.

(c) The primary key attributes are required to be (nonnull) and (unique).

(d) The (foreign key) specifies that the values of attributes for any record in the relation must correspond to values of the primary key attributes of some tuple in another relation.

(e) Subqueries that return only one tuple containing a single attribute are called (scalar subqueries).

(f) The (ORDER BY) clause causes the records in the result of a query to appear in sorted order.

(g) The (WITH) clause provides a way of defining a temporary relation whose definition is available only to the query in which the clause occurs.

(h) (Exercise problem 3.6) The SQL **LIKE** operator is case sensitive (in most systems), but the **LOWER**() function on strings can be used to perform case-insensitive matching. Show how to write a query that finds departments whose names contain the string "sci" as a substring, regardless of the case.

```
SELECT dept_name
FROM department
WHERE LOWER(dept_name) LIKE '%sci%';
```

(i) (Exercise problem 3.20) Show that, in SQL, **<> ALL** is identical to **NOT IN**.

Let S be the result of a subquery, and let x be a value. We compare the two expressions:

- x <> **ALL** S
- x **NOT IN** S

(1) x <> **ALL** $S \;\Rightarrow\; x$ **NOT IN** S

By definition, x <> **ALL** S means: $\forall s \in S,\ x \neq s$

This implies that there is no element in S equal to x.

Therefore, $x \notin S$.

Hence, x **NOT IN** S

(2) x **NOT IN** $S \;\Rightarrow\; x$ <> **ALL** S

By definition, x **NOT IN** S means: $x \notin S$

This implies $\forall s \in S,\ x \neq s$

Hence, x <> **ALL** S

Since both implication hold: x <> **ALL** $S \;\Leftrightarrow\; x$ **NOT IN** S.

Thus, in SQL,

$\therefore$ <> **ALL** $\equiv$ **NOT IN**

(j) (Exercise problem 3.19) List two reasons why null values might be introduced into the database.

Null values may be introduced into a database for the following reasons:
1. Missing or unknown information: A value may exist but is not currently known or available at the time of data entry.
2. Inapplicable attribute: A value does not exist because the attribute is not relevant for a particular tuple (e.g., a spouse field for an unmarried person).

2. (based on Exercise problems 3.9, 3.10, 3.16, and 3.17) Consider the relational database of Figure 3.19, where the primary keys are underlined. Given an expression in SQL for each of the following queries.

(a) Find the ID of each employee who does not work for "First Bank Corporation".
SQL Query:

```
SELECT ID
FROM works
WHERE company_name <> "First Bank Corporation";
```

(b) Find the ID, name, and city of residence of each employee who works for "First Bank Corporation" and earns more than $10,000.
SQL Query:

```
SELECT ID, name, city
FROM employee
WHERE ID IN (SELECT ID
             FROM works
             WHERE company_name = "First Bank Corporation" AND salary > 10000);
```

(c) Find the ID of each employee who earns more than every employee of "Small Bank Corporation".
SQL Query:

```
SELECT ID
FROM works
WHERE salary > ALL (SELECT salary
                    FROM works
                    WHERE company_name = "Small Bank Corporation");
```

(d) Assume that companies may be located in several cities. Find the name of each company that is located in every city in which "Small Bank Corporation" is located. Your query should run on MySQL.
SQL Query:

```
SELECT C.company_name
FROM company AS C
WHERE NOT EXISTS (
        SELECT city
        FROM company
        WHERE company_name = "Small Bank Corporation" AND city NOT IN (
                SELECT city
                FROM company AS D
                WHERE C.company_name = D.company_name
        )
);
```

(e) Find the name of the company that has the most employees (or companies, if there is a tie).
SQL Query:

```
SELECT company_name
FROM works
GROUP BY company_name
HAVING COUNT(DISTINCT ID) >= ALL (
        SELECT COUNT(DISTINCT ID)
        FROM works
        GROUP BY company_name
);
```

(f) Find the name of each company whose employees earn a higher salary on average, than the average salary at "First Bank Corporation".
SQL Query:

```
SELECT company_name
FROM works
GROUP BY company_name
HAVING AVG(salary) > (
        SELECT AVG(salary)
        FROM works
        WHERE company_name = "First Bank Corporation"
);
```

(g) Modify the database so that the employee whose ID is '12345' now lives in a city called "Newtown".
SQL Query:

```
UPDATE employee
SET city = 'Newtown'
WHERE ID = '12345';
```

(h) Find ID and name of employee who lives in the same city as the location of the company for which the employee works.
SQL Query:

```
SELECT ID, name
FROM employee E
WHERE E.city = (
        SELECT C.city
        FROM company C, works W
        WHERE W.company_name = C.company_name AND E.ID = W.ID
);
```

(i) Find ID and name of each employee who earns more than the average salary of all employees of her or his company.
SQL Query:

```
SELECT E.ID, name
FROM works AS W, employee AS E
WHERE E.ID = W.ID
AND salary > (
    SELECT AVG(salary)
    FROM works AS W2
    WHERE W.company_name = W2.company_name
);
```

(j) Find the company that has the smallest payroll (sum of all salary in a company).
SQL Query:

```
SELECT company_name
FROM works
GROUP BY company_name
HAVING SUM(salary) <= ALL ( SELECT SUM( salary )
                            FROM works
                            GROUP BY company_name );
```

(k) Given all employees of “First Bank Corporation” a 10 percent raise.
SQL Query:

```
UPDATE works
SET salary = salary * 1.1
WHERE company_name = ‘First Bank Corporation’;
```

(l) Delete all tuples in the works relation for employees of “Small Bank Corporation”.
SQL Query:

```
DELETE FROM works
WHERE company_name = ‘Small Bank Corporation’;
```

Working with MySQL:
Use a database client software to connect to the class DBMS that was introduced in the lectures. Below uses the "university" database (NOT *university_small*), which shares the same schemas with the database used in the lectures but contains a larger set of data records collected within a different period of time.

3. Find the answers to the following questions and provide the SQL queries showing how you find them. All queries should be complete to obtain the listed answers solely by themselves.

(a) List all instructor names in the *Accounting* department.

Answer:

```
+---------+
| name    |
+---------+
| Lembr   |
| Moreira |
| Hau     |
| Ullman  |
+---------+
```

SQL Query to obtain your answer:

```
SELECT name
FROM instructor
WHERE dept_name='Accounting';
```

(b) How many **students** are in the *Statistics* department?

Answer:

```
+----------+
| COUNT(*) |
+----------+
|       85 |
+----------+
```

SQL Query to obtain your answer:

```
SELECT COUNT(*)
FROM student
WHERE dept_name='Statistics';
```

(c) How many **unique student names** are in the *Astronomy* department?

Answer:

```
+----------------------+
| COUNT(DISTINCT name) |
+----------------------+
|                  104 |
+----------------------+
```

SQL Query to obtain your answer:

```
SELECT COUNT(DISTINCT name)
FROM student
WHERE dept_name='Astronomy';
```

(d) Find all students who have "**db**" as a substring in their name.

Answer:

```
+----------+
| name     |
+----------+
| Goldbu   |
| Sandberg |
| Sandberg |
+----------+
```

SQL Query to obtain your answer:

```
SELECT name
FROM student
WHERE name LIKE '%db%';
```

4. Find the answers to the following questions and provide the SQL queries showing how you find them. All queries should be complete to obtain the listed answers solely by themselves.

(a) (Exercise problem 3.26) For each student who has retaken a course at least twice (*i.e.*, the student has taken the course at least three times), show the course ID and the student's ID. Please display your results in order of course ID and do not display duplicate rows.

Answer:

```
+-----------+-------+
| course_id | ID    |
+-----------+-------+
| 362       | 16480 |
| 362       | 16969 |
| 362       | 27236 |
| 362       | 39925 |
| 362       | 39978 |
| 362       | 44881 |
| 362       | 49611 |
| 362       | 5414  |
| 362       | 69581 |
| 362       | 9993  |
+-----------+-------+
```

SQL Query to obtain your answer:

```
SELECT DISTINCT course_id, ID
FROM university.takes
GROUP BY ID, course_id
HAVING COUNT(*) > 2
ORDER BY course_id;
```

(b) Write a query that **lists up all classes** that have been open in the university, together **with the number of students** who were in each class. More specifically, enumerate all the *course IDs, section IDs, years, and semesters*, along with the *number of students who took each of the classes*.
Hint: you may want to come up with a result that starts as below.

course_id	sec_id	semester	year	num_students
105	1	Fall	2009	327
105	2	Fall	2002	307
137	1	Spring	2002	304
158	1	Fall	2008	291
158	2	Spring	2008	286
169	1	Spring	2007	300
169	2	Fall	2002	280
192	1	Fall	2002	338
200	1	Spring	2007	299
200	2	Fall	2002	292
237	1	Spring	2008	302
…	…	…	…	…

Query (you do not need to submit your query result):

```
SELECT course_id, sec_id, semester, year, COUNT(*) AS num_students
FROM takes
GROUP BY course_id, sec_id, semester, year;
```

(c) (Exercise problem 3.30) Consider the following SQL query on the university schema:

SELECT *AVG*(salary) - (*SUM*(salary)/*COUNT*(*))
FROM instructor;

We might expect that the result of this query is zero since the average of a set of numbers is defined to be the sum of the numbers divided by the number of numbers. Indeed, this is true for the example *instructor* relation in Figure 2.1. However, there are other possible instances of that relation for which the result would NOT be zero. Give one such instance, and explain why the results would not be zero.

Answer:

When one or more salary values are NULL (not available), the result may not be zero.
In SQL, aggregate functions such as SUM() and AVG() ignore NULL values, but COUNT(*) counts all tuples, including those with NULL values. Therefore, SUM(salary) is computed over only non-NULL salaries, whereas COUNT(*) counts all rows, including those with NULL salaries. As a result, the expression may not be zero, since the denominators differ.

Now connect to the class DBMS again and use the "university" database (<u>NOT *university_small*</u>). Find the answers to the following questions and provide the SQL queries showing how you find them. All queries should be complete to obtain the listed answers solely by themselves.

(d) Find the names of departments whose budget is higher than that of *Psychology*. List them in alphabetic order.

Answer:

```
+-----------+
| dept_name |
+-----------+
| Finance   |
| Physics   |
+-----------+
```

SQL Query to obtain your answer:

```
SELECT dept_name FROM department
WHERE budget > (SELECT budget
                FROM department
                WHERE dept_name='Psychology')
ORDER BY dept_name;
```

(e) List the names of the students in the *Geology* department whose name starting with 'C'.
Answer:

```
+-------------+
| name        |
+-------------+
| Collet      |
| Chakraborty |
| Cacciari    |
+-------------+
```

SQL Query to obtain your answer:

```
SELECT name
FROM student
WHERE dept_name = 'Geology' AND name LIKE 'c%';
```

(f) Write a query that counts the **number of students for each department** and sort the results in **descending order of the student counts**. *Hint: the head of the query result looks like the following:*

dept_name	num_students
Civil Eng.	120
Languages	119
History	117
Pol. Sci.	109
Comp. Sci.	108
...	...

Query (you do not need to submit your query result):

```
SELECT dept_name, COUNT(*) AS num_students
FROM student
GROUP BY dept_name
ORDER BY num_students DESC;
```

(g) (Exercise problem 3.22) Rewrite the WHERE clause

WHERE UNIQUE (SELECT title FROM course)

without using the UNIQUE construct.

Answer:

```
+-------------------------------------------------+
| title                                           |
+-------------------------------------------------+
| Diffusion and Phase Transformation              |
| Differential Equations                          |
| Antidisestablishmentarianism in Modern America  |
| Manufacturing                                   |
| Drama                                           |
  ...
```

SQL Query to obtain your answer:

```
# Possible sol 1
SELECT title
FROM course D
WHERE 1 >= (SELECT COUNT(*)
            FROM course C
            WHERE D.title = C.title);

# Possible sol 2
SELECT title
FROM course
WHERE title IN (SELECT title
                FROM course
                GROUP BY title
                HAVING COUNT(*) = 1);
```